



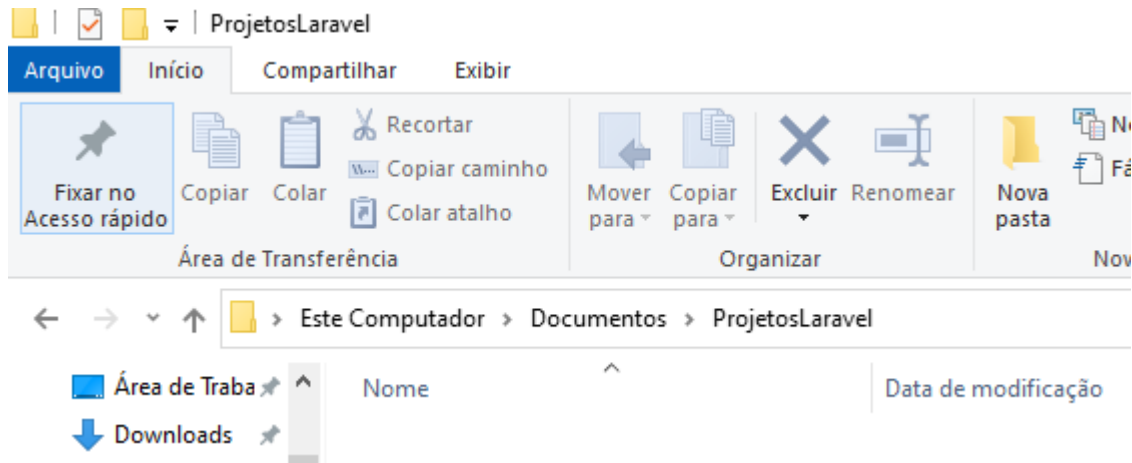
CRUD Simples

Tutorial Laravel – Parte 1

Paulo Roberto Tazinazo Cândido

Criação do Projeto

Criar pasta



Para acionar o servidor web embutido no Laravel para verificar se a aplicação foi criada corretamente e para testá-la durante o desenvolvimento, utilize:

php artisan serve

ctrl + c para interromper o servidor web..

Criar projeto (distribuição estável)

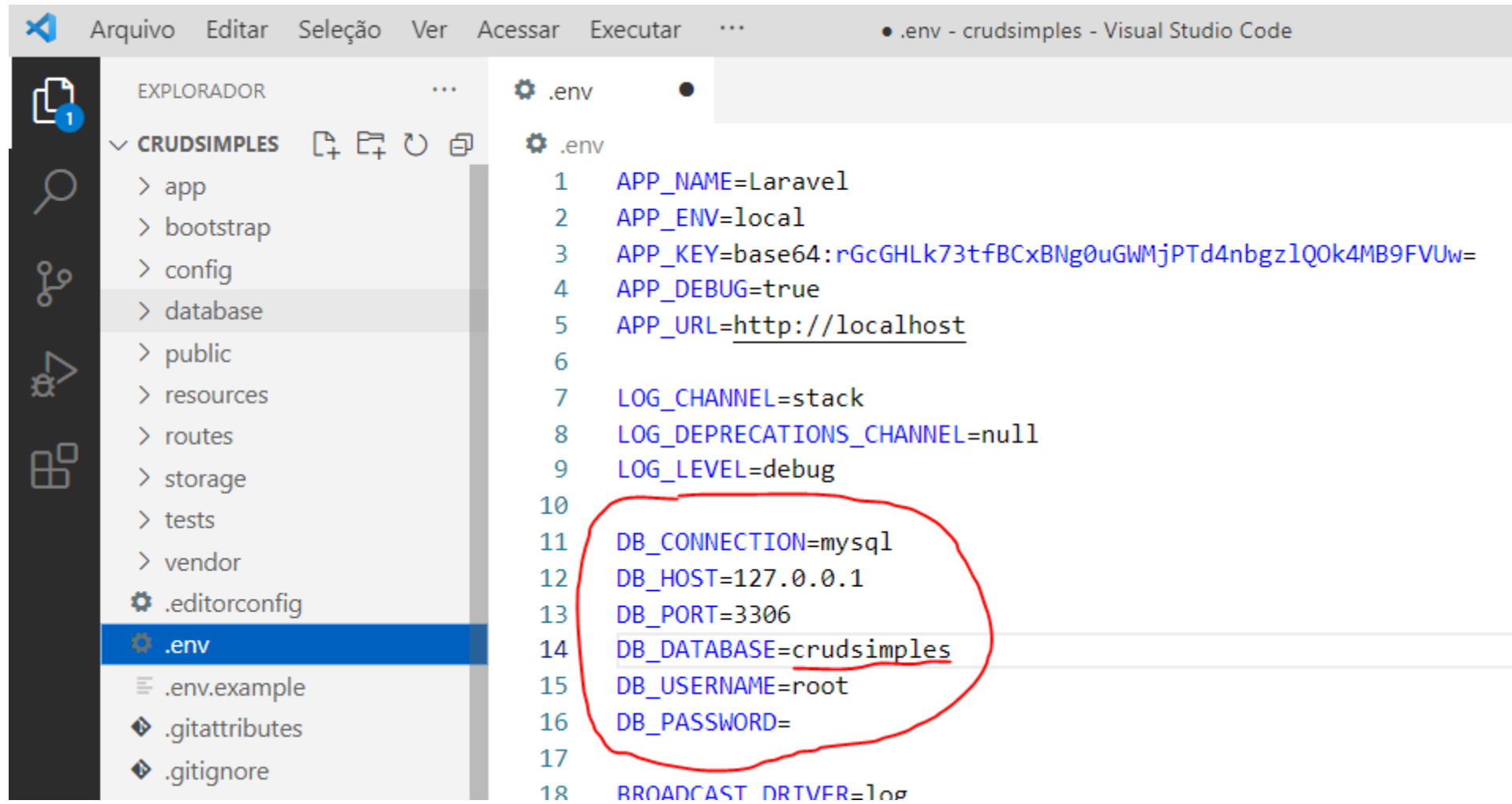
```
C:\Windows\System32\cmd.exe
Microsoft Windows [versão 10.0.19044.2604]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\paulocandido\Documents\ProjetosLaravel>composer create-project --prefer-dist laravel/laravel crudsimples
```

`--prefer-dist` tentaria baixar e descompactar os arquivos das dependências usando o GitHub ou outra API quando disponível. Isso é usado para download mais rápido de dependências na maioria dos casos. Ele não baixa todo o histórico VCS das dependências e deve ser melhor armazenado em cache. Além disso, os arquivos no GitHub podem excluir alguns arquivos que você não precisa apenas para usar a dependência com a diretiva de exclusão `.gitattributes`.

Fonte: Stack Overflow

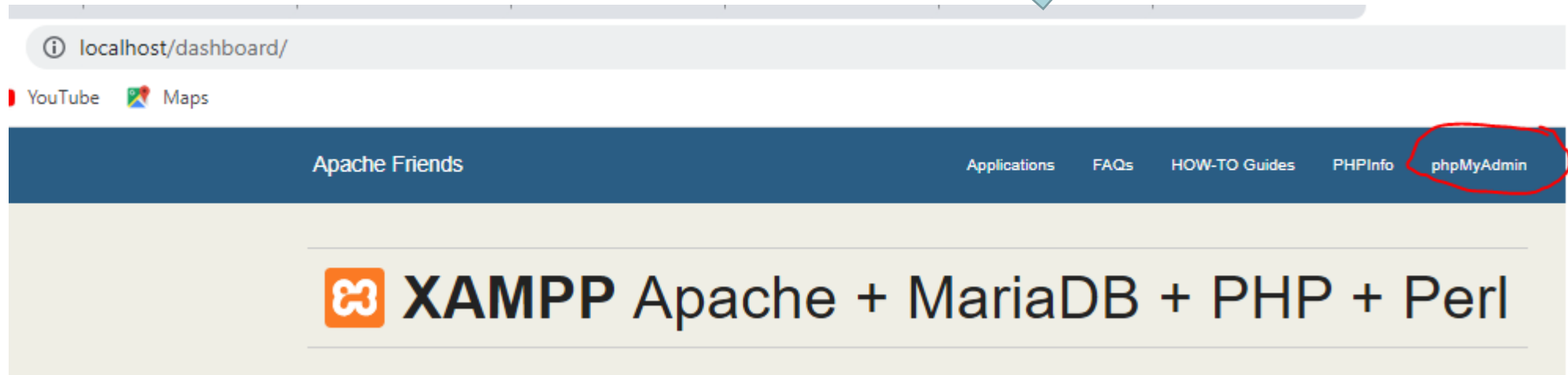
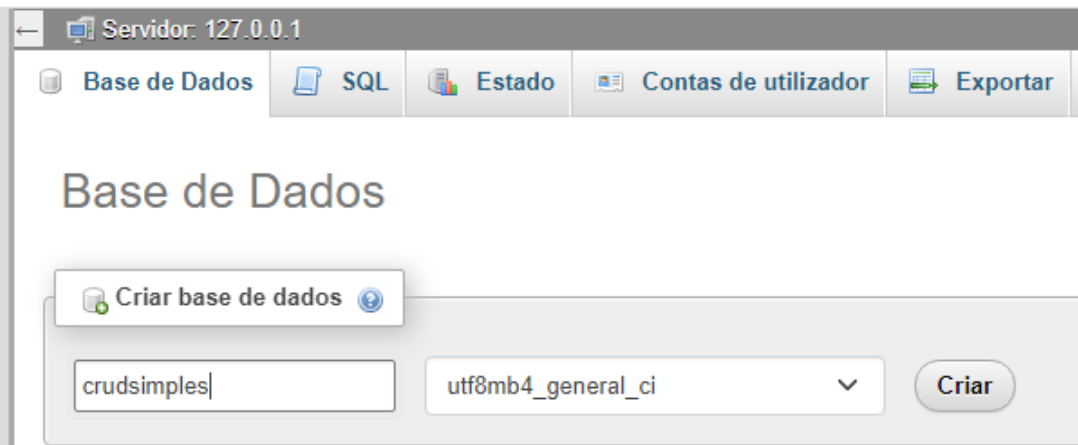
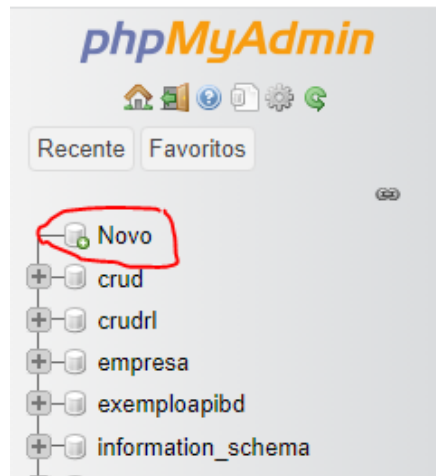
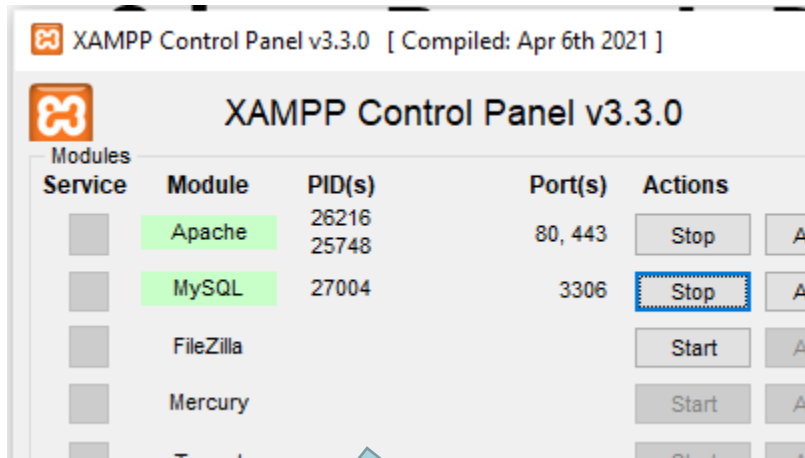
Configuração do Banco de Dados



The image shows the Visual Studio Code interface with the `.env` file open. The Explorer sidebar on the left shows the project structure for `CRUDSIMPLES`, with `.env` selected. The main editor displays the contents of the `.env` file, which includes various application settings. A red circle highlights the database configuration section, which is as follows:

```
1 APP_NAME=Laravel
2 APP_ENV=local
3 APP_KEY=base64:rGcGHLk73tfBCxBNg0uGWMjPTd4nbgz1QOk4MB9FVUw=
4 APP_DEBUG=true
5 APP_URL=http://localhost
6
7 LOG_CHANNEL=stack
8 LOG_DEPRECATIONS_CHANNEL=null
9 LOG_LEVEL=debug
10
11 DB_CONNECTION=mysql
12 DB_HOST=127.0.0.1
13 DB_PORT=3306
14 DB_DATABASE=crudsimples
15 DB_USERNAME=root
16 DB_PASSWORD=
17
18 BROADCAST_DRIVER=log
```

Criar o Banco de Dados



Criar os Componentes para CRUD

```
>php artisan make:model Computador --controller --resource --migration
```

O comando acima, além de criar o model Computador, cria o controller (parâmetro `--controller`) onde ficarão as funções (ou método, se preferir) usadas para desenvolver as funcionalidades de inserção de novo computador (C), consulta (R), alteração (U) e deleção (D).

Com o parâmetro `--resource` são criadas no controller as funções para o CRUD.

Com o parâmetro `--migration` é gerada a migração necessária para alterar o banco de dados criando a tabela que armazenará os dados dos objetos Computador.

Codificar o Model

Models são os objetos que contêm os dados da aplicação e, portanto, precisam ser armazenados em meio não volátil. Neste projeto é usado banco MySQL.

Única instrução a ser inserida no código gerado para o model é a definição do campo \$fillable, relacionando as propriedades dos objetos do tipo Computador.

É obrigatória a existência da propriedade 'id' em todos os models.

▼ CRUDSIMPLES

▼ app

> Console

> Exceptions

> Http

▼ Models

🐘 Computador.php

🐘 User.php

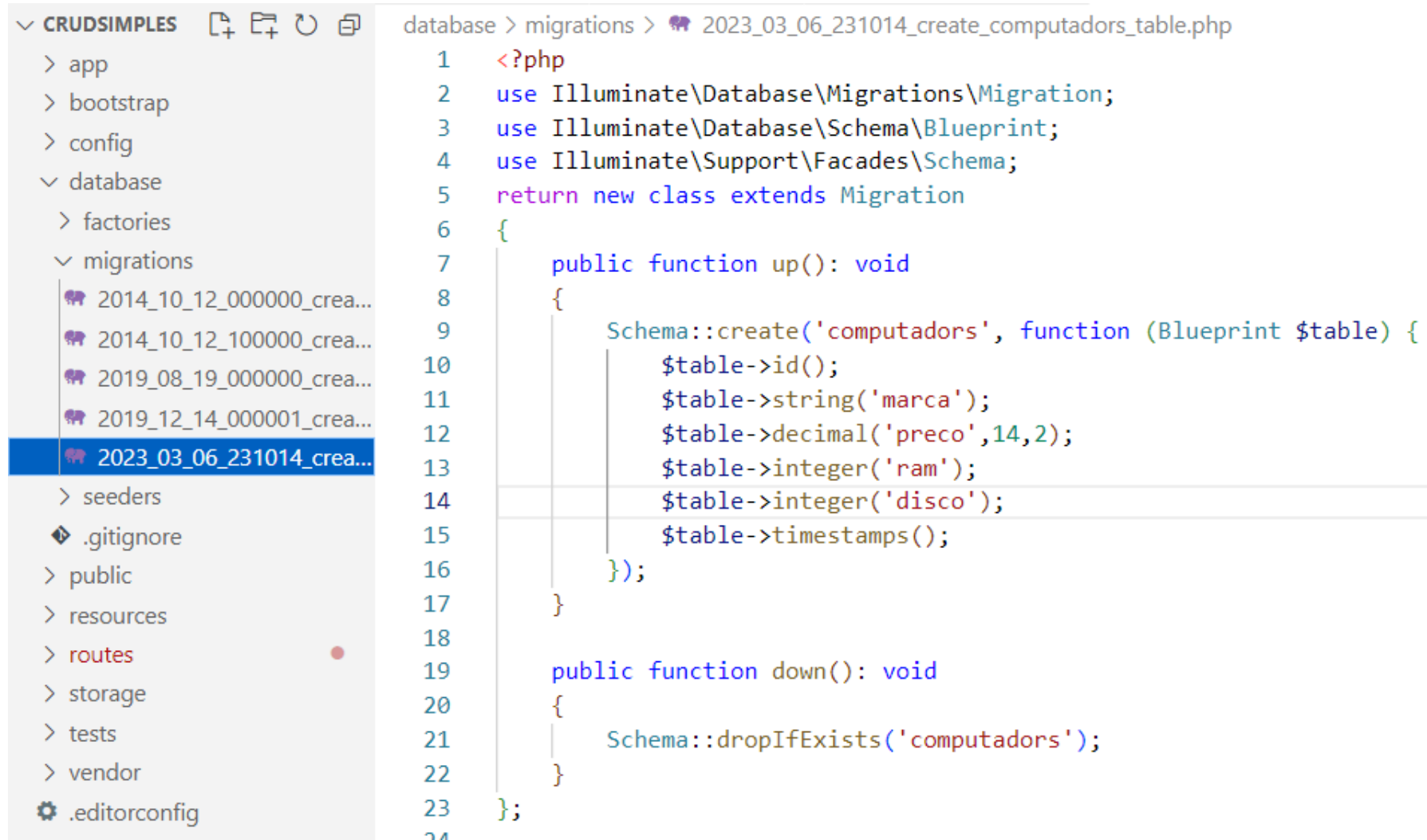
> Providers

app > Models > 🐘 Computador.php

```
1  <?php
2  namespace App\Models;
3  use Illuminate\Database\Eloquent\Factories\HasFactory;
4  use Illuminate\Database\Eloquent\Model;
5  class Computador extends Model
6  {
7      use HasFactory;
8      protected $fillable = ['id', 'marca', 'preco', 'ram', 'disco'];
9  }
10
```

Codificar migration

A migration, quando executada, criará a tabela “computadors” no banco de dados. Na migration definimos a estrutura de colunas da tabela onde serão armazenados os dados dos objetos “Computador” (model). Por convenção do Laravel, o nome da tabela corresponde ao nome do model no plural (todos os caracteres minúsculos).



```
database > migrations > 2023_03_06_231014_create_computadors_table.php
1  <?php
2  use Illuminate\Database\Migrations\Migration;
3  use Illuminate\Database\Schema\Blueprint;
4  use Illuminate\Support\Facades\Schema;
5  return new class extends Migration
6  {
7      public function up(): void
8      {
9          Schema::create('computadors', function (Blueprint $table) {
10              $table->id();
11              $table->string('marca');
12              $table->decimal('preco',14,2);
13              $table->integer('ram');
14              $table->integer('disco');
15              $table->timestamps();
16          });
17      }
18
19      public function down(): void
20      {
21          Schema::dropIfExists('computadors');
22      }
23  };
24
```

Executar migrações (migration) pendentes

```
>php artisan migrate
```

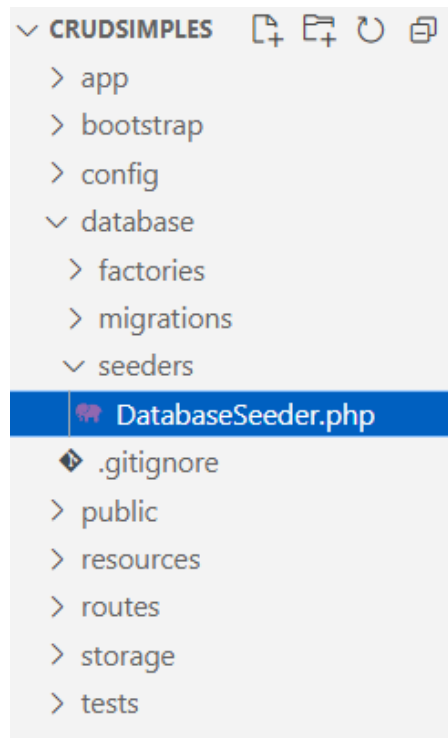
Certifique-se que o MySql esteja ativo.

```
INFO Preparing database.
Creating migration table ..... 27ms DONE
INFO Running migrations.
2014_10_12_000000_create_users_table ..... 40ms DONE
2014_10_12_100000_create_password_reset_tokens_table ..... 43ms DONE
2019_08_19_000000_create_failed_jobs_table ..... 29ms DONE
2019_12_14_000001_create_personal_access_tokens_table ..... 42ms DONE
2023_03_06_231014_create_computadors_table ..... 12ms DONE
```

As quatro migrações pré-definidas do Laravel são executadas junto com a que cria a tabela “computadors”.

Cadastrar computadores para testes

Pode haver a necessidade do banco de dados ser recriado em ambiente de desenvolvimento e todos os dados existentes serão perdidos. O Laravel inclui os Seeders para automatizar a carga de dados para realização de testes da aplicação. Codifique o DatabaseSeeder conforme abaixo para cadastrar três computadores usando a classe Computador e seus métodos herdados da classe Model fornecida pelo Laravel.



database > seeders > DatabaseSeeder.php

```
1  <?php
2  namespace Database\Seeders;
3
4  // use Illuminate\Database\Console\Seeds\WithoutModelEvents;
5  use Illuminate\Database\Seeder;
6  use App\Models\Computador;
7
8  class DatabaseSeeder extends Seeder
9  {
10     public function run(): void
11     {
12         Computador::create(['marca'=>'Lenovo','preco'=>5000,'ram'=>8,'disco'=>500]);
13         Computador::create(['marca'=>'Acer','preco'=>4800,'ram'=>8,'disco'=>500]);
14         Computador::create(['marca'=>'Samsung','preco'=>7200,'ram'=>16,'disco'=>1000]);
15     }
16 }
17
```

Para eliminar e recriar
todas as tabelas do
banco de dados, utilize:
php artisan migrate:fresh

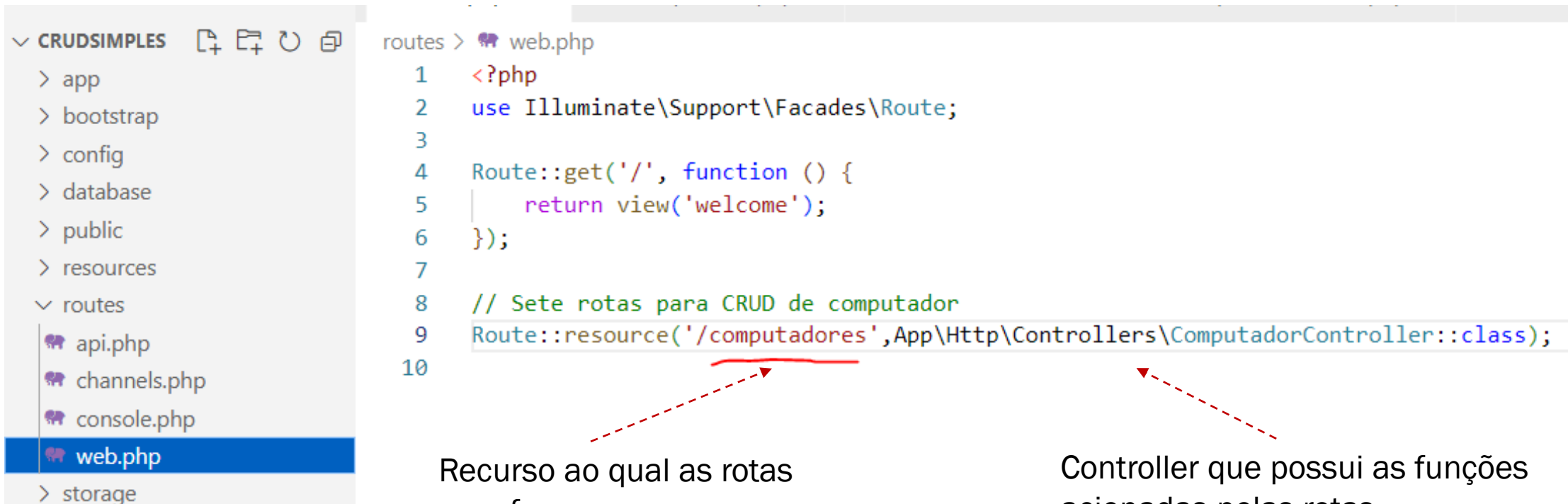
Recriar tabelas e exec. DatabaseSeeder:
php artisan migrate:fresh --seed

```
>php artisan db:seed
```

Executa o DatabaseSeeder

Definir as rotas para acionamento dos funções do Controller

No arquivo routes/web.php, inserir a instrução a seguir. Ela define as 7 rotas necessárias para o CRUD.



```
routes > web.php
1  <?php
2  use Illuminate\Support\Facades\Route;
3
4  Route::get('/', function () {
5      return view('welcome');
6  });
7
8  // Sete rotas para CRUD de computador
9  Route::resource('/computadores', App\Http\Controllers\ComputadorController::class);
10
```

Recurso ao qual as rotas se referem

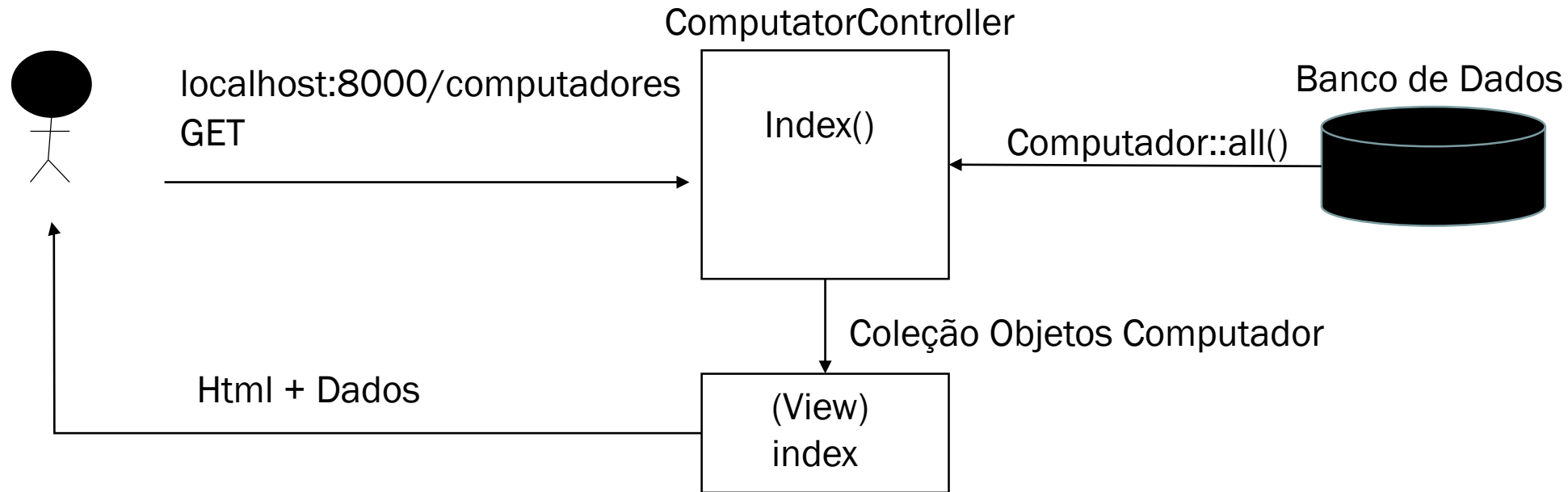
Controller que possui as funções acionadas pelas rotas.

php artisan route:list

```
GET|HEAD / ..... ignition.executeSolution > Spatie\LaravelIgnition > ExecuteSolutionController
POST _ignition/execute-solution ..... ignition.executeSolution > Spatie\LaravelIgnition > ExecuteSolutionController
GET|HEAD _ignition/health-check ..... ignition.healthCheck > Spatie\LaravelIgnition > HealthCheckController
POST _ignition/update-config ..... ignition.updateConfig > Spatie\LaravelIgnition > UpdateConfigController
GET|HEAD api/user .....
GET|HEAD computadores ..... computadores.index > ComputadorController@index
POST computadores ..... computadores.store > ComputadorController@store
GET|HEAD computadores/create ..... computadores.create > ComputadorController@create
GET|HEAD computadores/{computadore} ..... computadores.show > ComputadorController@show
PUT|PATCH computadores/{computadore} ..... computadores.update > ComputadorController@update
DELETE computadores/{computadore} ..... computadores.destroy > ComputadorController@destroy
GET|HEAD computadores/{computadore}/edit ..... computadores.edit > ComputadorController@edit
GET|HEAD sanctum/csrf-cookie ..... sanctum.csrf-cookie > Laravel\Sanctum > CsrfCookieController@show
```

Showing [13] routes

Codificar consulta a todos os computadores

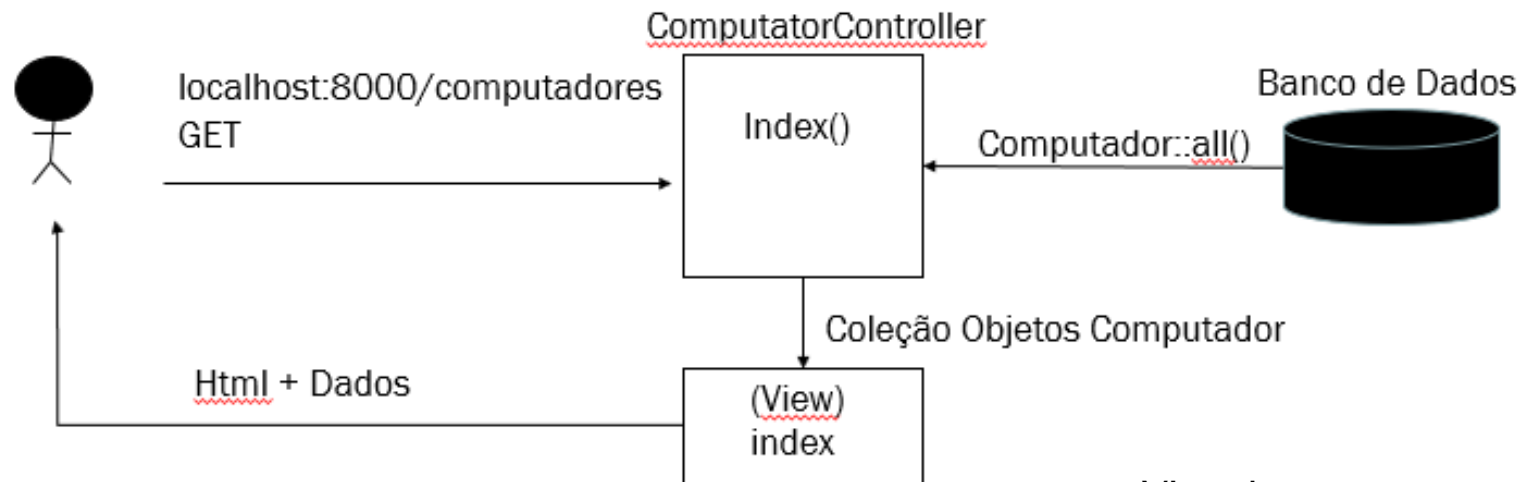


The screenshot shows a code editor with the following components:

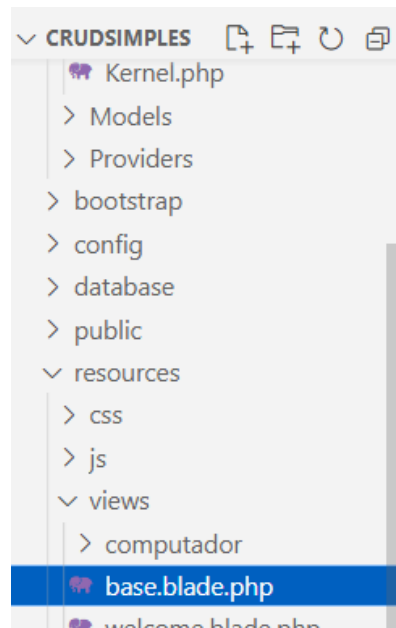
- EXPLORER:** A sidebar showing the project structure. The `ComputadorController.php` file is selected under the `Http > Controllers` directory.
- Code Editor:** The main area showing the implementation of the `index()` method in `ComputadorController.php`. The code is as follows:

```
1 <?php
2 namespace App\Http\Controllers;
3 use App\Models\Computador;
4 use Illuminate\Http\Request;
5 class ComputadorController extends Controller
6 {
7     public function index()
8     {
9         return View('computador.index')->with('ComputadorCollection',Computador::all());
10    }
11
12    public function create()
```
- Tab Bar:** At the top, several tabs are visible, including `web.php`, `ComputadorController.php` (the active tab), `Computador.php`, and a date tab `2023_03_06_231014_c`.
- Footer:** A small profile picture and the name `Prof. Charles Bonani` are visible in the bottom right corner.

Codificar consulta a todos os computadores



View base para construção das demais. Contém o código comum a qualquer documento Html

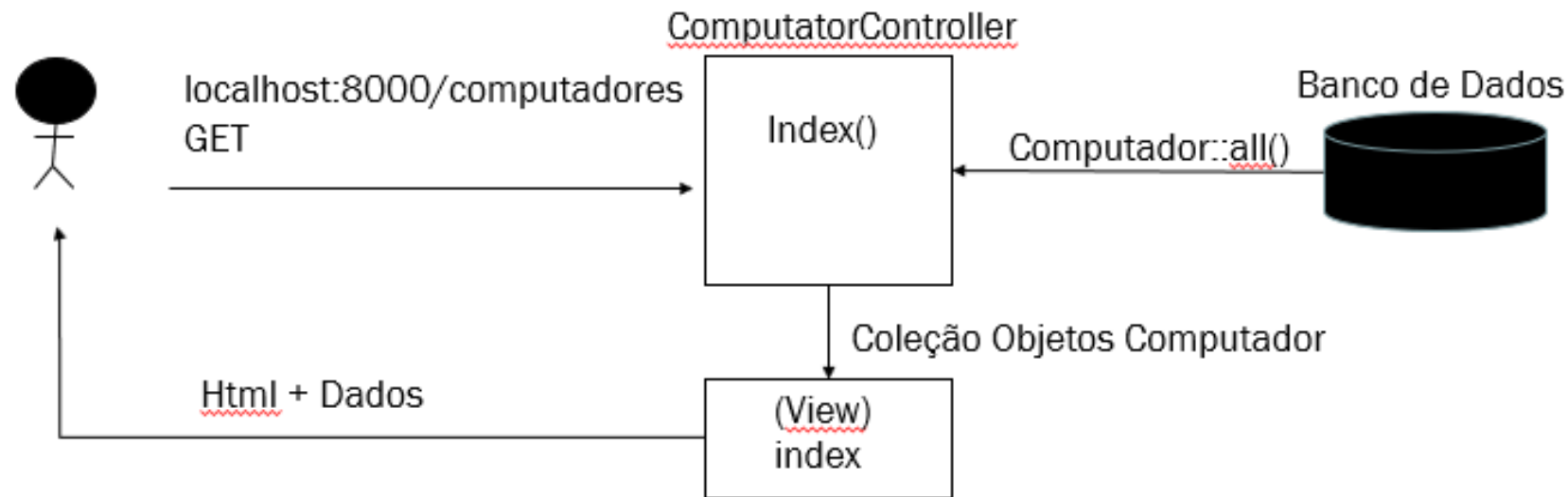


resources > views > base.blade.php

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <meta http-equiv="X-UA-Compatible" content="IE=edge">
6     <meta name="viewport" content="width=device-width, initial-scale=1.0">
7     <title>Document</title>
8   </head>
9   <body>
10    <!-- View filhas vão definir o conteúdo a ser colocado aqui -->
11    @yield('conteudo')
12  </body>
13 </html>
```

(*)

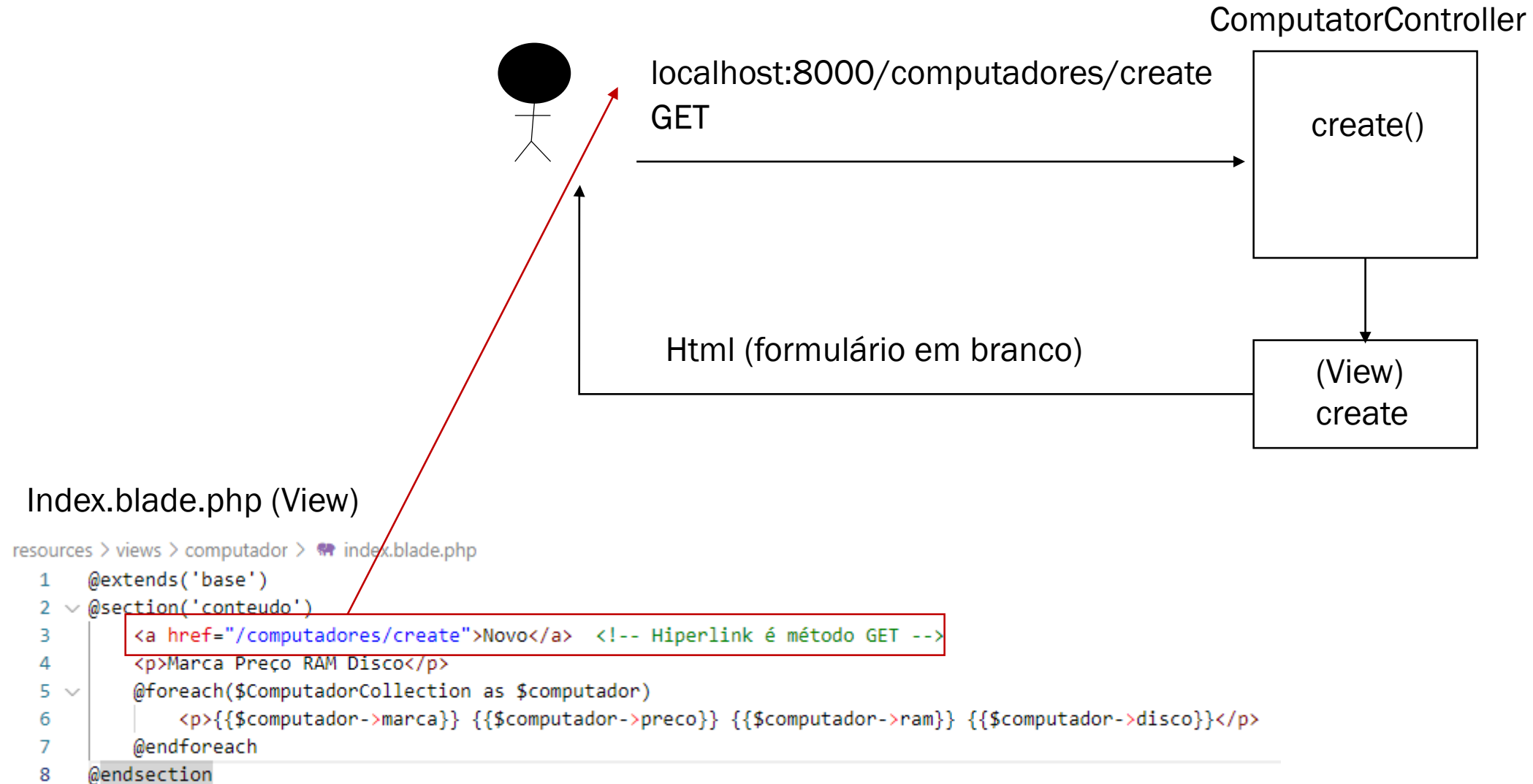
Codificar consulta a todos os computadores



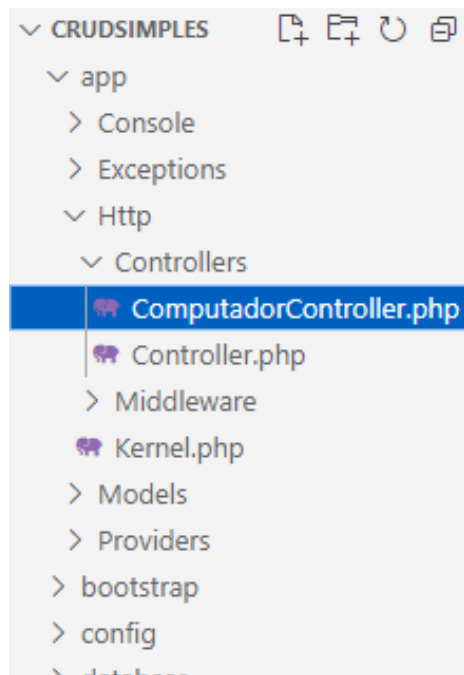
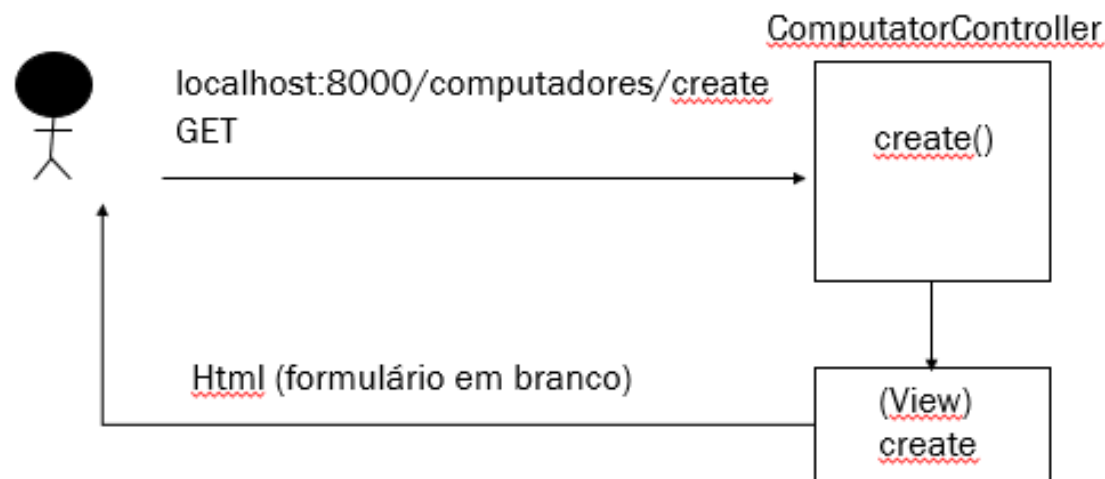
```
resources > views > computador > index.blade.php
1 @extends('base')
2 @section('conteudo')
3     <p>Marca Preço RAM Disco</p>
4     @foreach($ComputadorCollection as $computador)
5         <p>{{ $computador->marca }} {{ $computador->preco }} {{ $computador->ram }} {{ $computador->disco }}</p>
6     @endforeach
7 @endsection
```

View filha definindo o conteúdo do corpo do documento.

Codificar inclusão de novo computador



Codificar inclusão de novo computador



app > Http > Controllers > ComputadorController.php

```
1  <?php
2  namespace App\Http\Controllers;
3  use App\Models\Computador;
4  use Illuminate\Http\Request;
5  class ComputadorController extends Controller
6  {
7      public function index()
8      {
9          return View('computador.index')->with('ComputadorCollection',Computador::all());
10     }
11
12     public function create()
13     {
14         return View('computador.create');
15     }
16 }
```

Codificar inclusão de novo computador

View computador.create

Marca:

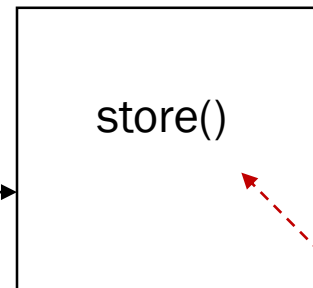
Preço:

RAM:

Disco:

[Voltar](#)

ComputatorController

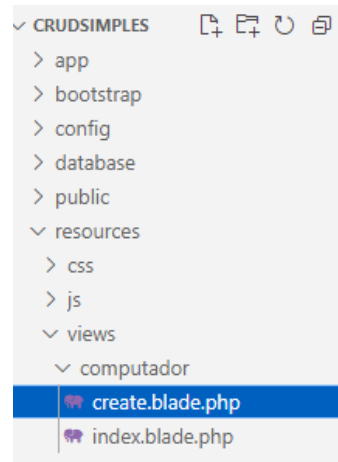


localhost:8000/computadores
POST

REDIRECT (relação de computadores)

localhost:8000/computadores
GET

View computador.create



resources > views > computador > create.blade.php

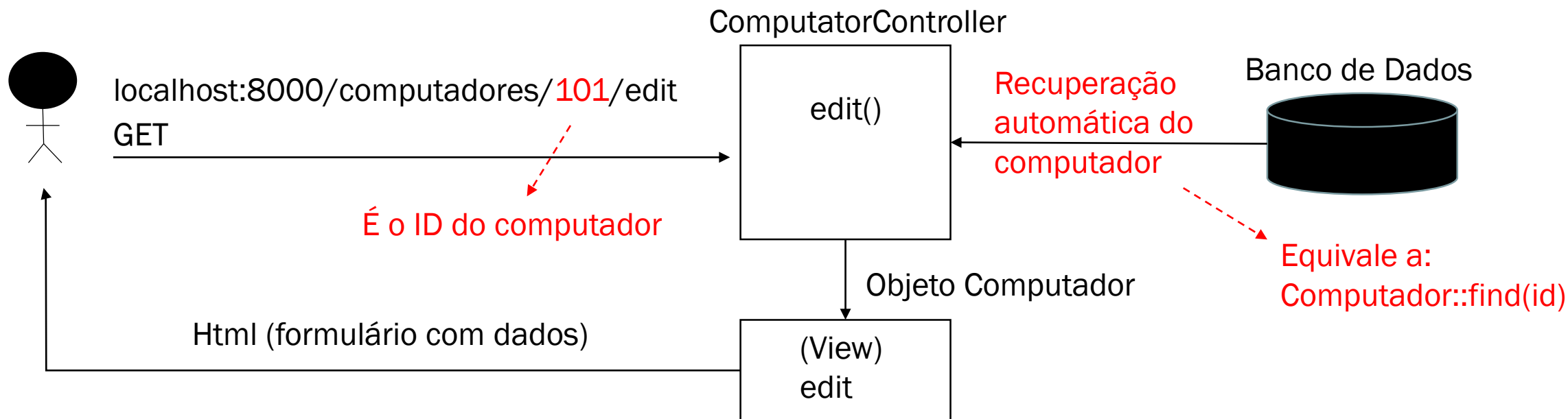
```
1 @extends('base')
2 @section('conteudo')
3     <form action="/computadores" method="post">
4         @csrf <!-- gera token de segurança verificado pela aplicação -->
5         <p>Marca: <input type="text" name="marca" /></p>
6         <p>Preço: <input type="number" name="preco" step="0.01" /></p>
7         <p>RAM: <input type="number" name="ram" /></p>
8         <p>Disco: <input type="number" name="disco" /></p>
9         <p>
10             <input type="submit" value="Enviar" />
11             <a href="/computadores">Voltar</a>
12         </p>
13     </form>
14 @endsection
```

ComputadorController

```
public function store(Request $request)
{
    // valida dados, visto na sequência
    // cria novo computador no banco de dados
    Computador::create( $request->all() );
    // $request->all() gerar vetor com os dados: ['marca'=>'Lenovo','preco'=>4500,...]
    return redirect('/computadores');
}
```

Cria novo objeto Computador no banco de dados usando os dados preenchido pelo usuário no formulário obtidos no objeto Request .

Codificar alteração de dados de computador



ComputadorController

```
//public function edit(Computador $computador)
public function edit(Computador $computadore)
{
    return View('computador.edit')->with('objetoComputador',$computadore);
}
```

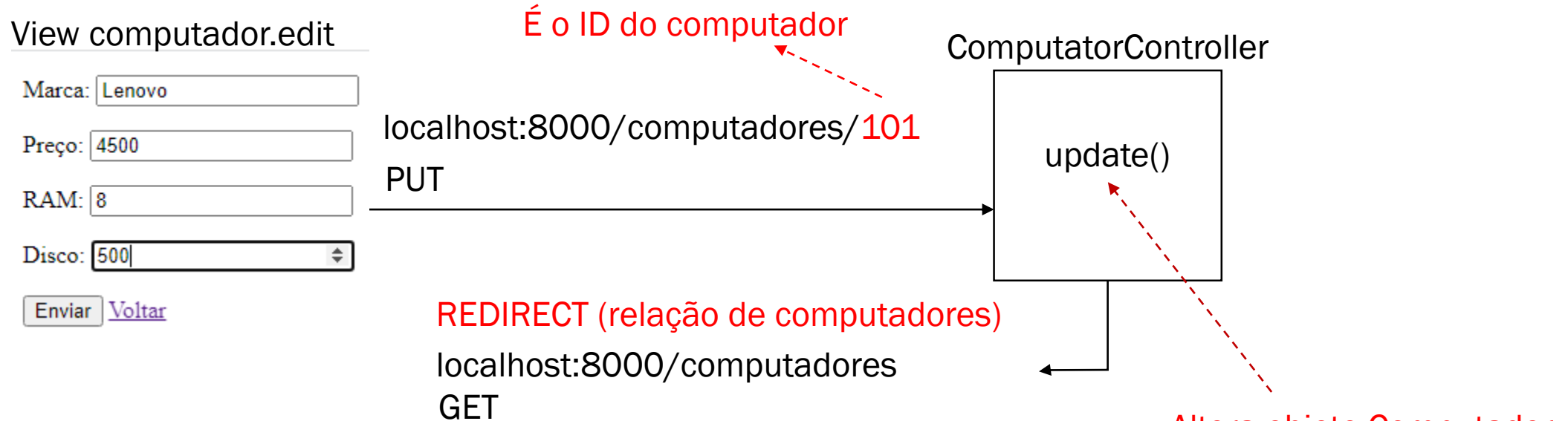
Automaticamente o objeto cujo id foi passado na URL é recuperado do banco de dados e colocado no parâmetro \$computadore, nome (definido na rota)

View computador.edit

```
@extends('base')
@section('conteudo')
    <form action="/computadores/{{ $objetoComputador->id }}" method="post">
        @method('put')
        @csrf <!-- gera token de segurança verificado pela aplicação -->
        <p>Marca: <input type="text" name="marca" value="{{ $objetoComputador->marca }}" /></p>
        <p>Preço: <input type="number" name="preco" step="0.01" value="{{ $objetoComputador->preco }}" /></p>
        <p>RAM: <input type="number" name="ram" value="{{ $objetoComputador->ram }}" /></p>
        <p>Disco: <input type="number" name="disco" value="{{ $objetoComputador->disco }}" /></p>
        <p>
            <input type="submit" value="Enviar" />
            <a href="/computadores">Voltar</a>
        </p>
    </form>
@endsection
```

Formulário submetido por post, mas para roteamento da requisição é usado put

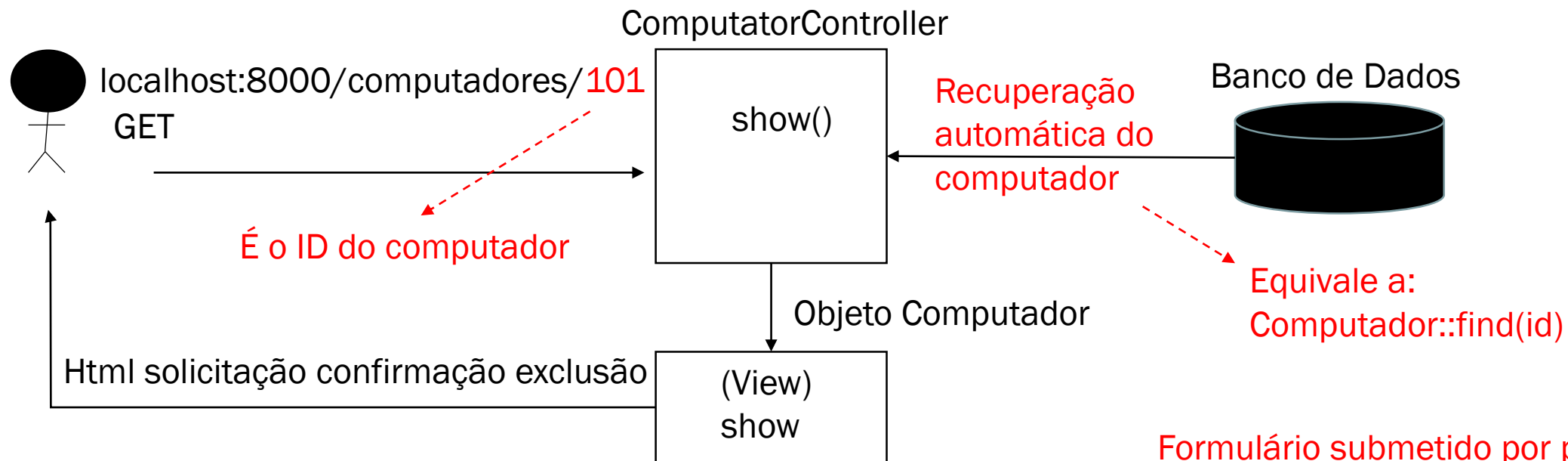
Codificar alteração de dados de computador



ComputadorController

```
public function update(Request $request, Computador $computadore)
{
    // validar dados, visto na sequência
    $computadore->update( $request->all() );
    // $request->all() gerar vetor com os dados: ['marca'=>'Lenovo', 'preco'=>4500,...]
    return redirect('/computadores');
}
```

Codificar exclusão de computador



ComputadorController

```
public function show(Computador $computadore)
{
    return View('computador.show')->with('objetoComputador', $computadore);
}
```

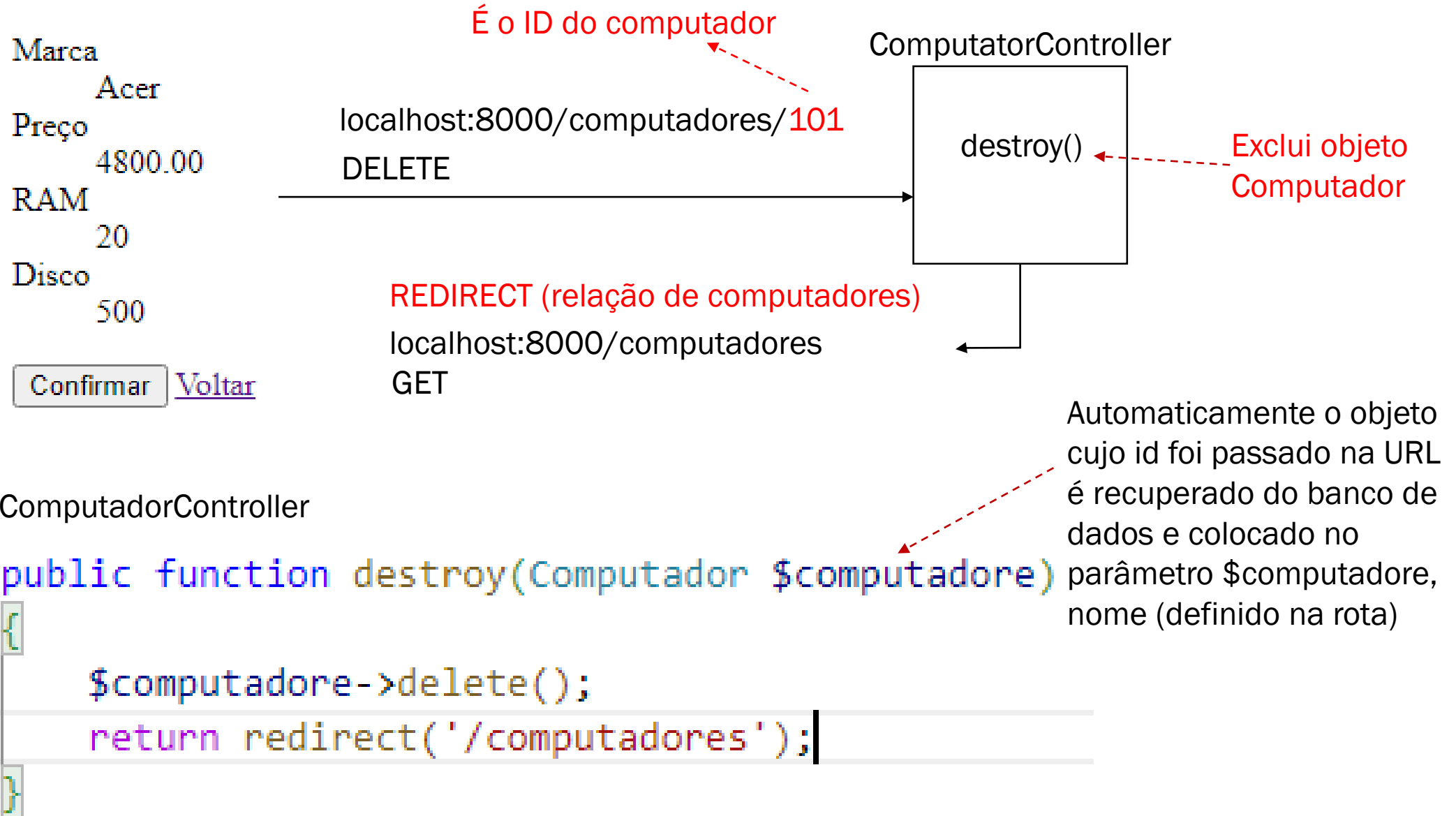
Automaticamente o objeto cujo id foi passado na URL é recuperado do banco de dados e colocado no parâmetro \$computadore, nome (definido na rota)

View computador.show

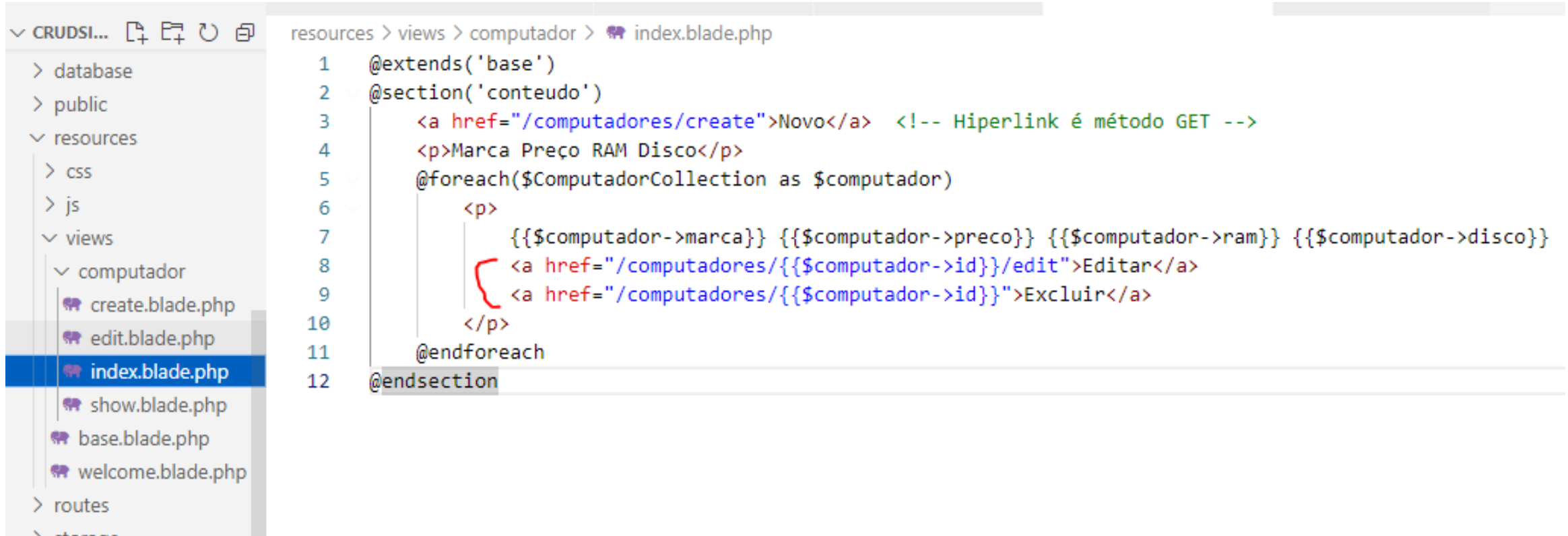
```
@extends('base')
@section('conteudo')
<dl>
    <dt>Marca</dt>
    <dd>{{ $objetoComputador->marca }}</dd>
    <dt>Preço</dt>
    <dd>{{ $objetoComputador->preco }}</dd>
    <dt>RAM</dt>
    <dd>{{ $objetoComputador->ram }}</dd>
    <dt>Disco</dt>
    <dd>{{ $objetoComputador->disco }}</dd>
</dl>
<form action="/computadores/{{ $objetoComputador->id }}" method="post">
    @csrf
    @method("delete")
    <input type="submit" value="Confirmar" />
    <a href="/computadores">Voltar</a>
</form>
@endsection
```

Formulário submetido por post, mas para roteamento da requisição é usado delete

Codificar exclusão de computador



Versão Final View Index



The image shows a code editor interface. On the left is a file explorer sidebar with the following structure:

- CRUDSI...
 - database
 - public
 - resources
 - css
 - js
 - views
 - computador
 - create.blade.php
 - edit.blade.php
 - index.blade.php**
 - show.blade.php
 - base.blade.php
 - welcome.blade.php
 - routes
 - storage

The main editor area displays the file path `resources > views > computador > index.blade.php` and the following code:

```
1 @extends('base')
2 @section('conteudo')
3     <a href="/computadores/create">Novo</a>  <!-- Hiperlink é método GET -->
4     <p>Marca Preço RAM Disco</p>
5     @foreach($ComputadorCollection as $computador)
6         <p>
7             {{ $computador->marca }} {{ $computador->preco }} {{ $computador->ram }} {{ $computador->disco }}
8             <a href="/computadores/{{ $computador->id }}/edit">Editar</a>
9             <a href="/computadores/{{ $computador->id }}">Excluir</a>
10        </p>
11    @endforeach
12 @endsection
```



Final Parte 1

Paulo Roberto Tazinazo Cândido